

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RK HEALTH COMPANION

AI-Powered Smart Patient Appointment & Medication Reminder System

An Internship Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

Submitted by

Mohan Sala

RK HEALTH INSTITUTE

July 2026

Declaration

I, Mohan Sala, hereby declare that the project report entitled “RK Health Companion – AI-Powered Smart Patient Appointment & Medication Reminder System” submitted to the Department of Computer Science & Engineering is an original work done by me. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree.

- **Date:** July 15, 2026
- **Signature of the Candidate:** Mohan Sala

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me throughout the course of this project. Their valuable suggestions, constructive feedback, and constant encouragement played an important role in helping me complete this work successfully.

I am also thankful to the institution for providing the necessary infrastructure, resources, and learning environment that made this project possible. Finally, I extend my heartfelt thanks to my family, friends, and peers for their continuous support and motivation during this work.

Abstract

- **Project Overview:** RK Health is a next-generation healthcare administration and patient reminder application that automates scheduling, monitors drug compliance, and interprets clinical records.
- **Problem:** Traditional patient portals lack automated reminder systems and fail to explain medical terminology, leading to poor medication compliance and patient confusion.
- **Solution:** A client-server application syncing clinical logs to Google Calendar, using a background scheduler for Twilio SMS reminders, and running Groq AI models to generate plain-text diagnosis summaries.
- **Technology:** Built using React 19, Express.js, Prisma, PostgreSQL (Supabase), Twilio, and the Groq API.
- **Outcome:** A fully secure web application that successfully tracks scheduling and medication compliance, improving overall clinical workflow.

Table of Contents

1. Introduction.....	7
2. Problem Statement.....	7
3. Objectives.....	7
4. Scope of the Project.....	8
5. Literature Survey	8
6. Feasibility Study	8
7. Software Development Methodology.....	9
8. System Requirements	9
9. Technology Stack	9
10. System Architecture.....	10
11. Project Directory Structure	10
12. Database Design	11
13. API Documentation	13
14. Authentication Flow.....	13
15. Module Documentation	14
16. UI Components	14
17. AI Module.....	14
18. SMS Reminder Module	15
19. Google Calendar Integration	15
20. Dashboard Analytics	15
21. Developer Guide	15
22. Security.....	16
23. Testing.....	17
24. Deployment	17
25. User Manual	18
26. Advantages.....	19
27. Limitations.....	19
28. Future Enhancements	19
29. Conclusion	19
30. References.....	20
31. Appendix — Abbreviations.....	20

1. Introduction

Modern healthcare systems face significant challenges in keeping patients engaged with their own care. Medication non-adherence and missed clinic appointments are widespread problems that lead to poor health outcomes, avoidable hospital readmissions, and billions of dollars in unnecessary costs worldwide.

Existing patient portals largely act as passive filing systems. They store medical records and appointment histories, but they do not actively remind patients of upcoming obligations, nor do they explain complex diagnoses in language a layperson can understand. RK Health Companion was built to close this gap. It offers a proactive, patient-friendly solution that automates reminders, simplifies diagnoses using artificial intelligence, and keeps every clinical visit and prescription organised in a single dashboard.

This report documents the complete design, development, and testing process undertaken during the internship, covering the problem definition, system architecture, database design, module-by-module functionality, security measures, testing strategy, and deployment pipeline.

2. Problem Statement

Patients frequently forget appointment times and medication schedules, particularly when managing multiple prescriptions with different dosages and timings. Manual tracking through paper notes or memory is error-prone and does not scale well for patients with chronic conditions who may be taking several medications a day.

In addition, clinical summaries handed to patients after a consultation are often written in dense medical jargon. This leaves patients confused about their own diagnosis, unsure of next steps, and less likely to comply with the prescribed treatment plan. RK Health Companion addresses both problems directly: automated reminders reduce the cognitive burden of tracking medications and appointments, while AI-generated plain-language summaries improve patient understanding and engagement.

3. Objectives

- **Primary Objective:** Develop an automated health manager that tracks scheduling and prescription compliance without requiring manual follow-up from patients.
- **Secondary Objective:** Provide simplified, plain-language medical summaries to improve patient health literacy.
- **Technical Objective:** Build a modular, responsive dashboard using React 19, TypeScript, and Express.js, with Prisma managing a PostgreSQL database hosted on Supabase.
- **AI Objective:** Use LLaMA 3.3-70B via the Groq API to translate clinical notes into plain English, structured for consistent rendering on the client.
- **Healthcare Objective:** Improve patient adherence to medications and reduce the rate of missed appointments through proactive, automated communication.

4. Scope of the Project

- **Current Scope:** Secure JWT-based login, appointment scheduling, Google Calendar sync, medication tracking, Twilio SMS reminders, Groq AI summaries, and report generation in PDF, Excel, and CSV formats.
- **Future Scope:** Video consultations, wearable-device integration, AI-based health predictions, a dedicated doctor portal, and prescription OCR (described further in Section 29).
- **Limitations:** Requires continuous internet connectivity and does not currently integrate with legacy hospital EHR/EMR databases.

5. Literature Survey

Before development began, a comparative survey of legacy Electronic Medical Record (EMR) systems and mainstream patient portals was carried out to identify functional gaps that RK Health Companion could address. The comparison below highlights the additional value delivered by the platform.

Feature	Legacy EMR	Typical Patient Portals	RK Health Companion
Database Storage	Yes	Yes	Yes (Supabase PostgreSQL)
Active SMS Alerts	No	Email Only	Yes (Twilio Integration)
Calendar Sync	No	Internal Only	Yes (Google Calendar API)
Clinical Translation	No	No	Yes (Groq LLaMA 3.3 AI)
File Export	PDF Only	PDF Only	PDF, Excel, CSV Export

This survey confirmed that while database-backed record storage is now standard, active, multi-channel patient engagement (SMS reminders, calendar sync, and AI-simplified explanations) remains rare, positioning RK Health Companion as a meaningfully differentiated solution.

6. Feasibility Study

- **Technical Feasibility:** Developed using React 19, Node.js, and Supabase; highly feasible with modern browsers and standard Node runtimes, with no proprietary hardware dependency.
- **Economic Feasibility:** Utilises open-source components (Prisma, PostgreSQL) and pay-as-you-go API tiers (Twilio, Groq, Google Developer Console), making it highly cost-effective for a project of this scale.
- **Operational Feasibility:** Features a clean, familiar dashboard layout that requires minimal technical training for patients to use effectively.
- **Schedule Feasibility:** The Agile methodology described in Section 7 allowed incremental delivery of working features within the internship timeframe, with each sprint producing a demonstrable increment.

7. Software Development Methodology

The project followed the Agile Software Development Life Cycle (SDLC), progressing through the following iterative phases. Each phase fed learnings back into the next, allowing the design to be refined as integrations were tested against real API behaviour.

- 1. Planning & Analysis
- 2. Database & Backend Design
- 3. API Implementation
- 4. Frontend UI Assembly
- 5. Third-Party Integrations
- 6. Testing & Quality Assurance
- 7. Production Deployment

8. System Requirements

Hardware

- Processor: Intel i5 or equivalent (minimum)
- RAM: 8 GB (minimum)
- Storage: 500 MB free space (minimum)

Software

- Operating System: Windows, macOS, or Linux
- Runtime Environment: Node.js v18.0.0 or higher
- Database Engine: PostgreSQL v15+ (Supabase)
- Web Browser: Chrome, Safari, Edge, or Firefox

9. Technology Stack

Layer	Technologies Used
Markup & Styling	HTML5, CSS3
Application Logic	JavaScript (ES6+), TypeScript
Frontend Framework	React 19
Build Tool	Vite
Routing	TanStack Router
Backend Runtime	Node.js, Express.js
ORM	Prisma Client
Database	Supabase (PostgreSQL)
AI Inference	Groq API (LLaMA 3.3-70B)
Notifications	Twilio SMS API
Calendar Integration	Google Calendar API

10. System Architecture

The platform follows a client-server architecture. The React frontend communicates with the Express.js backend over a secure REST API protected by JWT authentication. Every request first passes through CORS, Helmet, and JWT verification middleware before reaching a controller. The backend delegates database operations to Prisma, which queries the Supabase-hosted PostgreSQL database, and integrates with three external services for AI summaries, calendar sync, and SMS reminders.

- Patient Browser Client ↔ Vite Dev Server / Static Assets (UI, routing, styling)
- Patient Browser Client ↔ Express.js Application Server (REST requests over HTTPS)
- Express.js Server → JWT Auth Middleware (Verifies bearer token)
- Express.js Server ↔ Prisma Client ORM (Database actions)
- Prisma Client ORM ↔ Supabase PostgreSQL (SQL connection pool)
- Express.js Server → Groq LLaMA 3.3-70B API (AI prompt / summary requests)
- Express.js Server → Google Calendar API (Sync calendar events)
- Express.js Server → Twilio SMS Gateway (Background scheduled SMS)

11. Project Directory Structure

The codebase is organised as a monorepo-style layout with clearly separated backend and frontend packages, each responsible for a single concern:

```

rk-health-companion-main/
├── backend/
│   ├── src/
│   │   ├── config/           # Database & env parameter setup
│   │   ├── controllers/      # API route handlers & actions
│   │   ├── middleware/       # Auth guards, rate limits, error handlers
│   │   ├── prisma/           # Database migrations, seed & schema
│   │   ├── routes/           # Express route pathway bindings
│   │   ├── services/         # Groq, Google Calendar, Twilio gateways
│   │   ├── utils/            # Shared API wrapper helpers
│   │   ├── app.js             # Server application initialisation
│   │   └── server.js          # Port listener startup
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/       # UI layout components & modals
│   │   ├── lib/              # Custom state stores and API hooks
│   │   ├── routes/           # TanStack routing path screens
│   │   ├── styles.css         # Custom CSS variables & theming
│   │   └── main.tsx           # Root engine attachment point
│   ├── vite.config.ts
│   └── package.json

```

Core Authenticated Request Flow

1. Patient makes a request (e.g. POST /api/appointments)
2. Express applies the CORS, Helmet, and cookie-parser middleware chain
3. Express matches the route pattern and delegates control to the jwtVerify middleware

4. jwtVerify checks the Authorization: Bearer <token> header and attaches req.user
5. The controller processes the request, writes the database transaction via Prisma, and calls any third-party integration pipeline
6. The controller wraps the result in a standard successResponse JSON payload and returns it to the client

12. Database Design

RK Health uses PostgreSQL hosted on Supabase as its database engine, with schema mapping managed entirely through Prisma ORM. The relational model below captures how a single patient record connects to every other entity in the system.

Entity Relationships

- USER schedules many APPOINTMENT records
- USER tracks many MEDICATION records
- USER generates many REPORT records
- USER receives many NOTIFICATION records
- USER produces many ACTIVITY_LOG entries
- APPOINTMENT is evaluated by an optional AI_SUMMARY
- MEDICATION logs many REMINDER_HISTORY entries

User Table

Stores user account authentication, credentials, and medical health card profile.

Column	Type	Attributes	Description
id	String	PK, uuid	Unique user identifier
fullName	String	Not Null	User full name
email	String	Not Null, Unique	Registration email address
passwordHash	String	Not Null	Bcrypt hashed password
phone	String	Nullable	User phone number
dateOfBirth	DateTime	Nullable	User date of birth
gender	Enum	Nullable	Male, Female, Other
bloodGroup	String	Nullable	Blood type indicator
height / weight	String	Nullable	Height in cm / weight in kg
bmi	Float	Nullable	Calculated Body Mass Index
allergies	String	Nullable	Text checklist of allergies
medicalConditions	String	Nullable	Registered chronic conditions
insurance	String	Nullable	Active health insurance details

Column	Type	Attributes	Description
createdAt	DateTime	default(now)	User registration timestamp

Appointment Table

Stores clinical visits scheduled by patients.

Column	Type	Attributes	Description
id	String	PK, uuid	Appointment identifier
userId	String	FK (User.id)	Linked patient
doctorName	String	Not Null	Doctor name
title	String	Not Null	Visit description
appointmentDate	DateTime	Not Null	Date of clinical visit
appointmentTime	String	Not Null	Time of day (HH:MM)
visitType	Enum	Consultation, etc.	Visit category type
priority	Enum	Low, Medium, High	Priority rank indicator
status	Enum	Upcoming, Completed	Current appointment status
calendarLink	String	Nullable	Synced Google Calendar link

Medication Table

Stores patient prescriptions, dosage schedules, and compliance metrics.

Column	Type	Attributes	Description
id	String	PK, uuid	Prescription identifier
userId	String	FK (User.id)	Linked patient
medicineName	String	Not Null	Name of medicine
dosage	String	Not Null	Dosage size (e.g. 1 pill)
startDate / endDate	DateTime	Not Null	Prescription course range
reminderTime	String	Not Null	Daily trigger time (HH:MM)
status	Enum	Pending, Taken	Active daily taking status

Report Table

Maintains a historical record of reports generated by each user.

Column	Type	Attributes	Description
id	String	PK, uuid	Report identifier
userId	String	FK (User.id)	Linked user
title	String	Not Null	User-facing report title

Column	Type	Attributes	Description
reportType	String	Nullable	File format extension
reportPath	String	Not Null	Server filepath destination
generatedAt	DateTime	default(now)	Output timestamp

13. API Documentation

Every protected endpoint requires a Bearer JWT token supplied in the Authorization header. Key endpoint groups are summarised below.

Endpoint	Method	Auth	Description
/api/auth/register	POST	None	Registers a new patient account.
/api/auth/login	POST	None	Authenticates a user and returns a JWT.
/api/appointments	GET	Bearer	Retrieves a paginated, filterable list of appointments.
/api/appointments	POST	Bearer	Creates a new clinical appointment.
/api/medications	GET	Bearer	Retrieves the patient's medication list and compliance.
/api/medications	POST	Bearer	Adds a new medication with a reminder schedule.
/api/dashboard	GET	Bearer	Retrieves core dashboard data.
/api/dashboard/stats	GET	Bearer	Retrieves summary statistic cards.
/api/reports/generate	POST	Bearer	Generates a new report (PDF, Excel, or CSV).
/api/reports/:id/:format	GET	Bearer	Downloads a previously generated report file.

Sample Registration Response

```
POST /api/auth/register -> 201 Created
{
  "success": true,
  "message": "User registered successfully.",
  "data": { "id": "user-uuid-12345", "role": "patient" }
}
```

14. Authentication Flow

1. Patient Client sends POST /api/auth/login with email and password
2. Auth Controller queries the Supabase database for the matching user record
3. Database returns the user record, including the password hash
4. Auth Controller validates the submitted password against the hash using bcrypt
5. On success, the Auth Controller returns a success response containing a signed JWT
6. The client stores the token and attaches it as a Bearer token on subsequent requests

15. Module Documentation

Module	Function
Landing Page	Marketing and access gateway.
Login / Register	Secure access using JWT and Bcrypt encryption.
Dashboard	Core control panel displaying metrics, compliance charts, and timelines.
Appointments	Scheduling, updating, and deleting clinical appointments.
Medications	Medication list and daily compliance checklist.
AI Summary	Explains clinical notes in plain English.
Reports	Configures, generates, and downloads records in PDF, Excel, and CSV.
Activity Logs	Audits security actions and user operations.
Profile	Editing of contact and health information.

16. UI Components

The frontend is built with React 19 and TanStack Router, using a small set of reusable components:

- **AppLayout:** Consistent sidebar and content wrapper across all pages.
- **Topbar:** Greets the user and surfaces system notifications.
- **Modal Component:** Shared form layout for adding appointments, medications, and reports.
- **Theme Provider:** Supports light and dark theme switching.

17. AI Module

AI Summary Workflow

1. Doctor submits visit notes
2. Controller retrieves the clinical text
3. A structured prompt is built for LLaMA 3.3
4. The prompt is sent via a POST request to the Groq API
5. The JSON response is parsed
6. The result is saved to the AiSummary table
7. The summary is rendered on the client screen

Prompt Engineering

Clinical notes are sent to Groq with explicit instructions to translate medical terminology into plain English and to return a structured JSON object containing four fields: visitOverview, medicalExplanation, medicationInstructions, and followUpAdvice. Constraining the model to a fixed JSON shape keeps the frontend rendering logic simple and predictable.

18. SMS Reminder Module

- **Scheduling:** A Node-Cron job runs every minute to scan the Medication table.
- **Trigger:** If a medication's reminder time matches the current system time and falls within the prescription's date range, the system calls the Twilio API to send an SMS.
- **Log History:** Each reminder's status (Sent, Delivered, or Failed) is recorded in the ReminderHistory table.

```
cron.schedule("* * * * *", async () => {
  // Polls the database every minute for medications
  // matching the current reminder time and sends SMS
  // notifications via the Twilio REST API.
});
```

19. Google Calendar Integration

- **Workflow:** Creating an appointment automatically invokes createCalendarEvent.
- **API Call:** The backend uses the Google APIs client library, authenticated with a service account, to insert the event.
- **Save Link:** The resulting calendarLink is saved on the appointment record so users can open the event directly in Google Calendar.

```
const calendar = google.calendar({ version: "v3", auth: authClient });
```

20. Dashboard Analytics

The dashboard aggregates statistics to calculate medication compliance rates by comparing total logged doses against doses marked as Taken. Recharts is used to visualise compliance and appointment trends over time, giving patients an at-a-glance view of their adherence history.

21. Developer Guide

This section summarises the internal engineering conventions followed while building and maintaining the platform.

State Management Strategy

The client application does not treat local component state as the source of truth. PostgreSQL remains the single source of truth for all data; the frontend keeps subscribable cache copies of REST responses using React's useSyncExternalStore inside a custom store module. CRUD actions trigger a hydration call that refetches the relevant collection and notifies subscribed components, avoiding full page refreshes.

```
async add(item) {
  const payload = mapAppointmentToBackend(item);
```

```
const res = await apiFetch("/appointments", "POST", payload);
await this.hydrate(); // Refetches items, notifies React nodes
return mapAppointmentToFrontend(res.data);
}
```

API Request Wrapper

A shared `apiFetch` helper centralises three concerns: it automatically appends the user's JWT under a Bearer authorization header, resolves the base host (falling back to the local development server or a production proxy path), and parses error responses into a consistent shape that UI components can catch and display.

22. Security

Security was treated as a first-class requirement throughout development, given the sensitivity of healthcare data handled by the platform.

Identity & Authentication

- **Password Encryption:** Passwords are hashed with Bcrypt using 12 salting rounds during registration; plain-text passwords are never stored.
- **Route Authorization:** Protected Express routes are wrapped in a `jwtVerify` middleware that validates the bearer token before any controller logic executes.

```
const decoded = jwt.verify(token, env.JWT_SECRET);
req.user = await prisma.user.findUnique({ where: { id: decoded.id } });
```

Server Hardening

- **Helmet.js:** Configures secure HTTP response headers to defend against XSS, clickjacking, and MIME-sniffing attacks.
- **CORS Constraints:** Restricts API calls to explicitly allowed client origins.
- **Rate Limiting:** Restricts request volume per client IP to mitigate denial-of-service attempts (e.g. 100 requests per 15-minute window).

Database & Network Security

- **Connection Pooling:** Queries run through PgBouncer transaction pooling to prevent connection exhaustion under load.
- **Parameterised Queries:** Prisma Client generates parameterised SQL statements, eliminating SQL-injection risk.
- **Encryption In Transit:** Supabase enforces TLS/SSL encrypted channels for all data transactions.
- **Secrets Management:** Credentials such as JWT secrets, database URLs, and third-party API keys are stored as private environment variables and never committed to source control.

23. Testing

The backend test suite is written using Jest and Supertest to validate database schemas, controllers, routing paths, and API integrations.

```
backend/tests/
├── auth.test.js      # Register, duplicate-email check, login, JWT auth
├── settings.test.js  # GET/PUT user preferences & configuration
└── health.test.js    # Runtime checks, database connectivity status
```

Test Cases Matrix

Test Suite	Target	Input	Expected Output	Status
auth.test.js	User Registration	Valid details	201 Created, valid JSON	PASS
auth.test.js	Registration	Duplicate email	400 Bad Request error	PASS
settings.test.js	GET Preferences	Valid token	Default preferences object	PASS
settings.test.js	PUT Preferences	Updated config	Config persisted & echoed back	PASS
health.test.js	Database Health	GET /health	200 OK, DB connected status	PASS
health.test.js	Version Check	GET /version	Matches current semver tag	PASS

Run Commands

```
cd backend
npm run test          # Run the full Jest suite
npm run test:coverage # Generate an HTML coverage report
```

24. Deployment

Step 1 — Database (Supabase PostgreSQL)

1. Create a Supabase project and choose the database region and password
2. Copy the pooled connection string (port 6543) as DATABASE_URL and the direct connection string (port 5432) as DIRECT_URL
3. Deploy Prisma migrations to the hosted database with `npx prisma migrate deploy`

Step 2 — Backend API (Render)

The following environment variables are configured in the hosting provider dashboard:

Variable	Purpose
PORT / NODE_ENV	Server port and runtime environment
DATABASE_URL / DIRECT_URL	Supabase pooled and direct connection strings
JWT_SECRET	Secure signing key for authentication tokens
GROQ_API_KEY	Groq Developer Console API key for AI summaries

Variable	Purpose
TWILIO_ACCOUNT_SID / AUTH_TOKEN / PHONE_NUMBER	Twilio SMS credentials
GOOGLE_APPLICATION_CREDENTIALS	Service account JSON for Calendar sync

Build command: `npm install` Start command: `npm start`. Health is validated by calling `/api/health`, which should return status 200 with the database connectivity marked healthy.

Step 3 — Frontend Client (Vercel)

- **API URL:** `VITE_API_URL` is set to the deployed backend URL (e.g. `https://api.rkhealth.com/api`).
- **Build Target:** Build command `npm run build`, output directory `dist`.
- **Routing Fallback:** A `vercel.json` rewrite ensures `index.html` is served for all subroutes, since TanStack Router relies on client-side history.

25. User Manual

This section summarises the end-user workflow, from account creation through to generating a downloadable clinical report.

1. Registration & Authentication

1. Navigate to the application URL and select Sign In, or Create an Account if new
2. Enter Full Name, Email Address, and a secure Password, then submit
3. Log in with the registered credentials to reach the Dashboard

2. Navigating the Dashboard

The dashboard presents metrics cards (total appointments, today's medications, compliance rate, reports generated), a timeline feed of upcoming visits and notifications, and an interactive checklist for marking medications as taken.

3. Scheduling Clinical Appointments

1. Open Appointments in the sidebar and select New Appointment
2. Enter doctor name, title, date & time, priority, and visit type
3. Save — the appointment syncs to the database and, if enabled, to Google Calendar

4. Logging Medication Prescriptions

1. Open Medications and select New Medication
2. Enter medicine name, dosage, daily frequency, reminder time, and the start/end dates of the course
3. Save — the background cron job begins sending SMS reminders at the scheduled time

5. Viewing AI Appointment Summaries

For a completed appointment, selecting AI Summary triggers the Groq-powered breakdown: a clinical overview, a plain-language medical explanation, a medication guide, and next-step advice.

6. Compiling Reports

1. Open Reports and select Generate Report
2. Choose which modules to include and the desired format (PDF, CSV, or Excel)
3. Generate — the report compiles, downloads automatically, and is listed in the reports table

26. Advantages

- Proactive reminders sent directly to the patient's phone.
- Simplified medical jargon through AI translation.
- Seamless scheduling with automatic Google Calendar sync.
- Comprehensive report exports in multiple formats.
- Single source of truth architecture keeps the dashboard consistent across devices.

27. Limitations

- Requires a stable internet connection to access the dashboard and database.
- SMS and AI summaries are subject to third-party API rate limits.
- Does not integrate directly with hospital EHR systems.

28. Future Enhancements

- **OCR Reader:** Scan prescriptions and automatically extract medication schedules.
- **Wearables Integration:** Sync data with fitness trackers to monitor vital signs.
- **Video Consultations:** Secure, in-app video calling with doctors.
- **SOS Emergency Alert:** One-click button to contact emergency services.
- **Dedicated Doctor Portal:** A companion interface for clinicians to review patient compliance and update visit notes directly.

29. Conclusion

RK Health Companion is a secure healthcare manager that addresses medication non-compliance and missed-appointment issues. By combining automated SMS reminders, Google Calendar synchronisation, and LLaMA-based AI diagnostics, the platform helps patients stick to their care plans and improves overall health management. The project also served as a comprehensive practical exercise in full-stack development, covering secure API design, relational database modelling, third-party service integration, automated testing, and cloud deployment.

30. References

- Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, 2017.
- React 19 Documentation – react.dev
- Express.js Web Framework – expressjs.com
- Prisma ORM Reference – prisma.io/docs
- Supabase API Reference – supabase.com/docs
- Groq API Integration – console.groq.com/docs
- Twilio REST API Reference – twilio.com/docs

31. Appendix — Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
EHR	Electronic Health Record
JWT	JSON Web Token
ORM	Object Relational Mapping
PK / FK	Primary Key / Foreign Key
REST	Representational State Transfer
SMS	Short Message Service
UI / UX	User Interface / User Experience
SDLC	Software Development Life Cycle
ORM	Object-Relational Mapping
CORS	Cross-Origin Resource Sharing
OCR	Optical Character Recognition